Module Bases des moteurs de jeux (Unreal Engine)

Travaux Pratiques 1

Prise en main d'*Angular*, première application, liaisons des données

---

Objectif: L'objectif de cette séance est dans un premier temps d'installer l'environnement d'*Angular* et de créer une première application. Ensuite nous verrons les mécanisme de liaison de données défini entre les fichiers *TypeScript* et les fichiers *template*.

Partie I: Installation

Avant de commencer à construire une première application basée sur *Angular*, il s'agit d'installer les différents outils liés à *Angular*. En particulier, il sera tout d'abord nécessaire d'installer *Node.JS* afin d'avoir les fonctionnalités *Javascript* coté serveur. En effet, l'application sera réalisée et testée localement. Le déploiement d'une application sera traité dans une autre séance.

1.1 Vérifiez que *Node.JS* est installé sur votre machine. Pour cela ouvrez un terminal et tapez `node` et dans le cas contraire installez le programme à partir de l'adresse suivante : `https://nodejs.org/en/download`.

1.2 Testez le bon fonctionnement de *Node.JS* en récupérant le code exemple ci-dessous : (accessible à l'adresse suivante `https://nodejs.org/en/about` (image (a)) ou sur moodle.

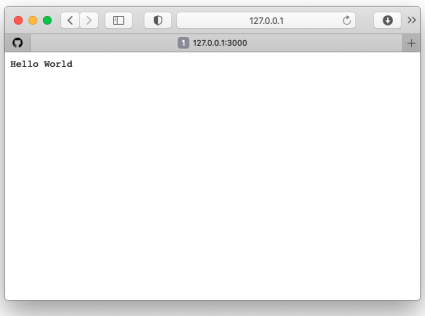
```
const http = require('http');

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

(a)

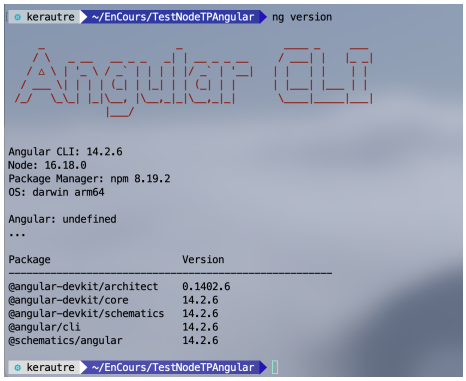


(b)

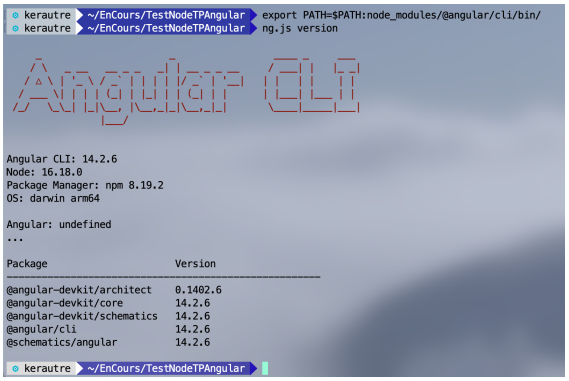
Une fois le code récupéré dans un fichier `testNode.js`, vous pouvez lancer le serveur dans un terminal puis vérifiez que la page est bien accessible localement. Vous devez pouvoir lancer un navigateur pour vérifier le contenu (comme sur l'image (b) ci-dessus).

```
node testNode.js
```

1.3 Pour vérifier si *Angular* est installé sur vos machines, vous pouvez écrire la commande `ng version` dans un terminal. Si tout est bien installé vous devriez avoir l'image (a) ci-dessous.



(a)



(b)

Si elle n'est pas présente et que vous avez les droits administrateur (cela doit être le cas sur vos machines), vous pouvez l'installer globalement :

```
npm install -g @angular/cli
```

Dans le cas contraire, il est toujours possible d'effectuer une installation locale en écrivant dans un terminal la ligne de commande suivante :

```
npm install @angular/cli
```

Ensuite vous pourrez avoir accès en recopiant le chemin utilisé sur l'image (b) ci-dessus.

1.4 Toujours dans un terminal, utilisez les commandes vues en cours pour créer une application de test (choisir un nom de projet temporaire et appliquer les options par défaut) :

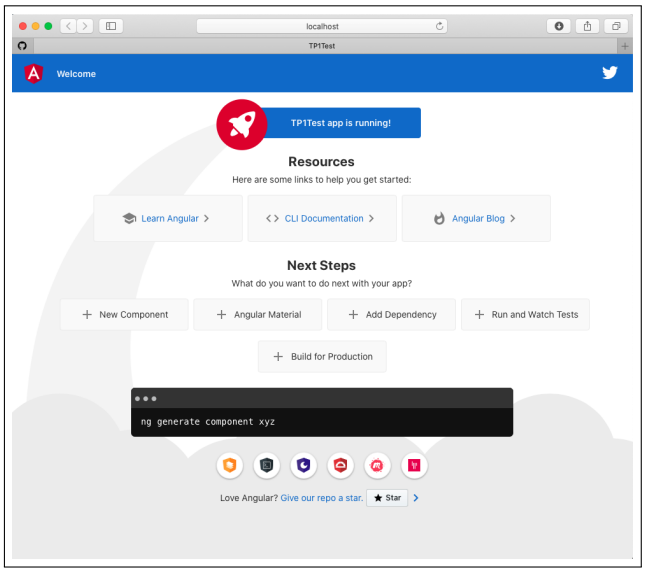


(a) génération projet



(b) lancement du projet

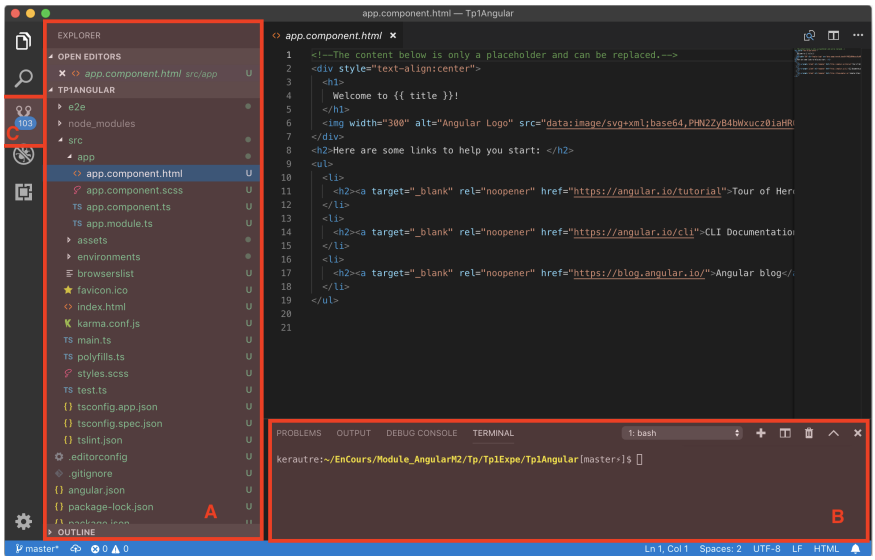
Si tout se passe correctement, vous devriez avoir une nouvelle page de votre navigateur qui se lance avec l'affichage suivant :



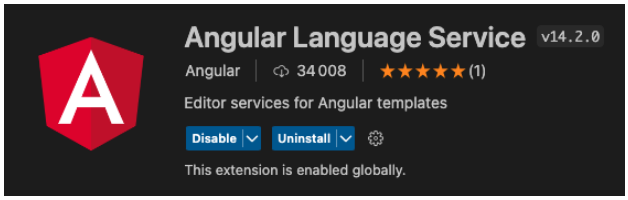
Pour la suite de cette séance vous pourrez utiliser l'éditeur de code de votre choix mais les exemples des codes seront donnés à partir l'éditeur gratuit déjà installé *Visual Studio Code*. Idéalement vous pouvez aussi choisir d'utiliser Visual Studio Code ou VSCodium qui est le même logiciel sans les ajouts de microsoft en utilisant le binaire VSCodium :

`https://vscodium.com`

Cet éditeur présente l'avantage de pouvoir s'installer à la fois sous *Windows*, *Linux* ou *MacOS*. Il permet aussi d'avoir une vie globale avec à la fois la visualisation de l'arborescence de l'application (panneau A) mais aussi l'accès à l'interpréteur CLI (panneau B). Enfin, il permet également d'exploiter git (onglet C).



1.5 Pour ouvrir votre projet, vous pouvez directement ouvrir le répertoire du projet ou faire un glissé-déposé du répertoire dans l'éditeur. Ouvrez votre projet et activez l'extension *Angular Langage service* si l'éditeur vous le propose.



1.6 Ouvrez la fenêtre du terminal de façon à obtenir la même visualisation que pour la question précédente, et lancez à nouveau le projet après l'avoir arrêté dans le terminal initial (utiliser la touche CTR-C pour arrêter le programme).

Partie II: Première application et premier composant

Après avoir installé les différents composants nécessaires pour créer une application *Angular*, nous allons créer notre propre application ainsi que notre premier composant qui aura pour rôle la gestion d'affichage de liste d'étudiants.

2.1 Il s'agit tout d'abord de créer le nouveau projet associé à cette séance de TP. Pour cela, comme précédemment, vous utiliserez la commande `ng new` en sélectionnant les options par défaut et le style SCSS :

```
ng new Tp1Angular
```

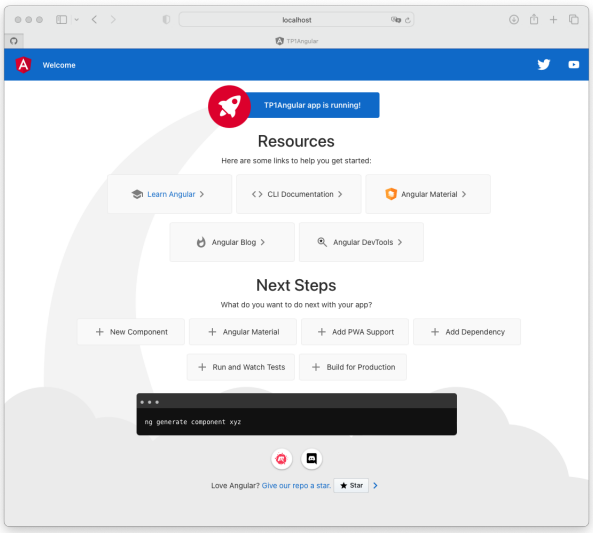
2.2 Ensuite afin de pouvoir utiliser une mise en page améliorée des pages générées, installer *Bootstrap* dans votre projet. Pour ce faire, dans un terminal, depuis le répertoire de votre projet (`Tp1Angular`) saisir la commande suivante :

```
npm install bootstrap
```

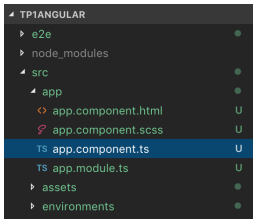
2.3 Pour intégrer *Bootstrap* dans votre projet il faut ajouter dans le fichier de configuration la ligne suivante 49 du fichier `angular.json`, dans les sections *architect*,*build*, *styles* :

```
30            "styles": [
31              "node_modules/bootstrap/dist/css/bootstrap.css",
32              "src/styles.scss"
33            ],
```

2.4 Sauvegardez bien votre fichier et lancez le serveur avec la commande `ng serve`. Si tout se passe bien vous devriez voir le style de la page légèrement différent au nouveau des couleur de lien et devrait désormais ressembler à l'image suivante :



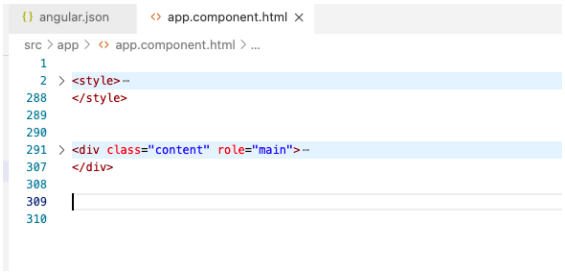
2.5 Explorez les différentes structures du projet et repérez le fichier permettant de changer le contenu de la page.



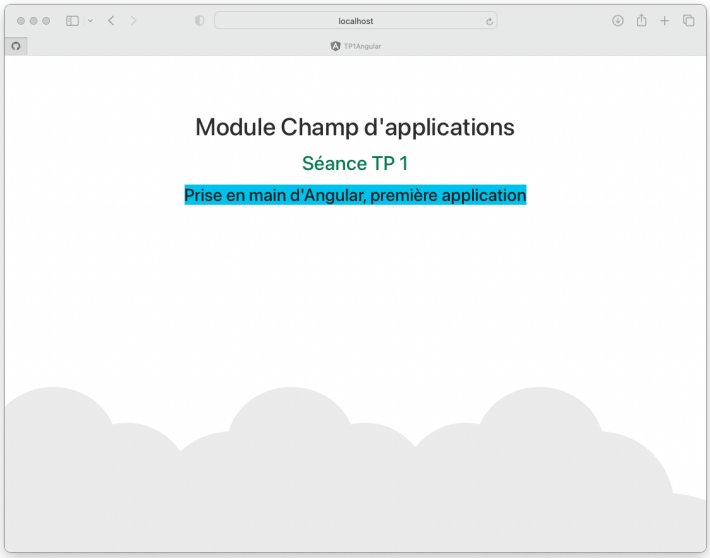
2.6 Modifiez le contenu de la page de façon à avoir la page suivante (image (b)) avec les contraintes ci-dessous :

— Le nom de la séance (**Séance 1**) sera défini à travers une variable.
— Le résumé de la séance sera aussi représenté dans une variable.
— Utiliser les classes *bootstrap* : `bg-info text-dark`, `text-success`.

Indication : vous pouvez supprimer le contenu en gardant les structures représentées sur l'image (a).

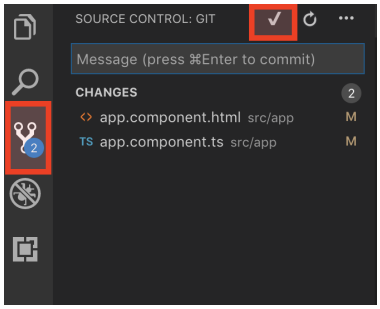


(a)



(b)

2.7 Utilisez *git* afin de commiter toutes les modifications effectuées à chaque question (voir image ci-dessous).

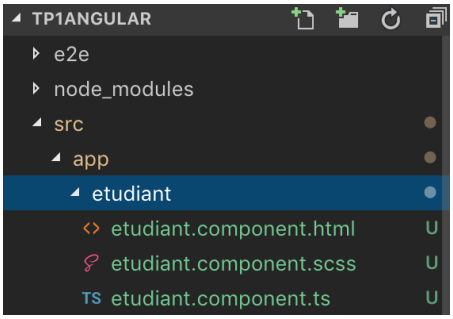


Maintenant que nous avons aperçu la structure principale d'une application, il s'agit de créer son propre composant.

2.8 Rajoutez un nouveau composant que vous nommerez `etudiant`. Pour rajouter un composant, vous utiliserez la commande suivante (a) et vous devriez obtenir les nouveaux fichiers suivants (image (b)) :

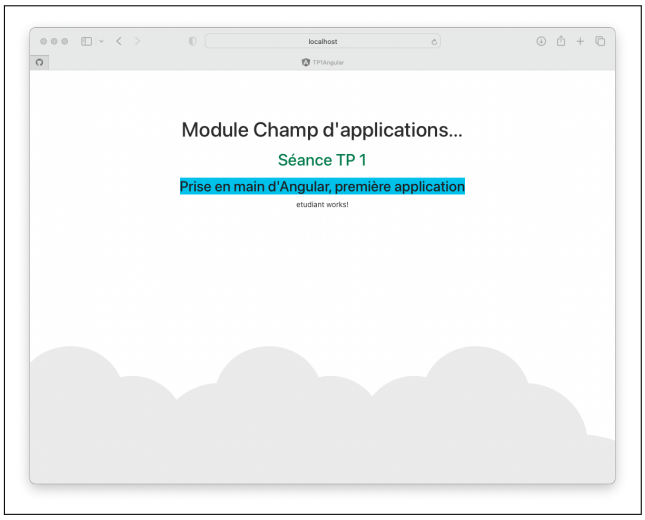
```
$ ng generate component
```

(a)



(b)

2.9 En observant la logique du contenu du fichier `index.html`, ajouter votre nouveau composant en modifiant le fichier `app.component.html`. Si tout fonctionne correctement vous devriez obtenir la page suivante :



Partie III: Gestion des données dynamiques

Après avoir fait une première application et un premier composant, il est important de connaître les mécanismes permettant de gérer les données de manière dynamique. Plus précisément, il s'agira dans cette partie, de considérer les mécanismes suivants :

- *String* interpolation.
- Liaisons par propriétés (*Property binding*).
- Liaisons par événements (*Event binding*).
- Liaisons par propriétés personnalisées.

3.1 Editez le fichier `etudiant.component.html` et rajoutez le style suivant de façon à obtenir un meilleur affichage :

```
<li class="card card-small">
  <h4>Ceci est Etudiant</h4>
</li>
```

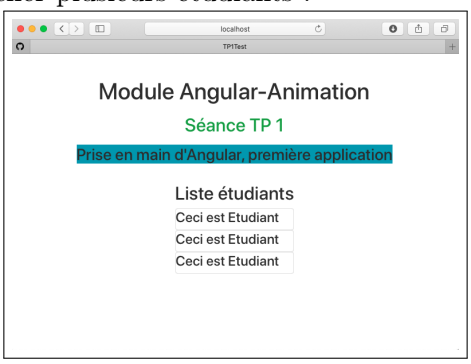
3.2 Pour afficher une liste en exploitant *Bootstrap*, dans le fichier `app.component.html`, utilisez le code suivant (a) associé à une liste de type `unordered list` de façon à obtenir l'affichage représenté sur l'image (b) ci-dessous. le code suivant qui permet d'afficher plusieurs étudiants :

```
<div class="card-container">
  <div class="row">
    <div class="col-xs-12">
      <h2>Liste étudiants</h2>
      Completer ici



    </div>
  </div>
</div>
```

(a)

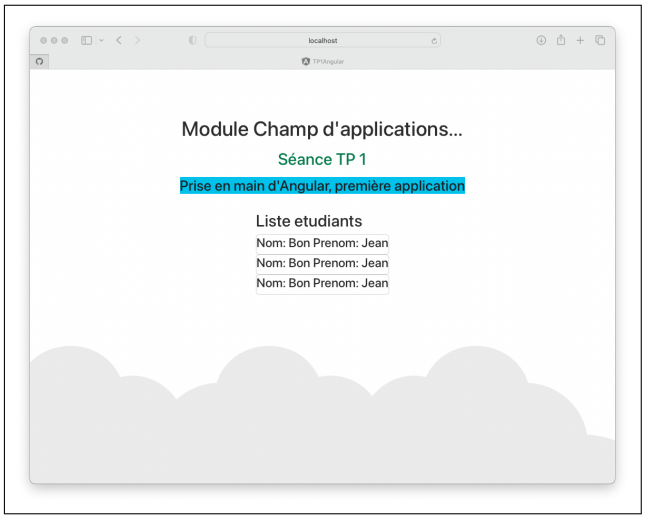


(b)

Pour l'instant, l'application ne présente pas encore de caractéristiques intéressantes par rapport à *Angular*, mais nous allons intégrer dans un premier temps un exemple de ***String* interpolation**.

Comme nous l'avons aperçu dans les questions précédentes, l'interpolation s'effectue en utilisant les symboles `{{ }}`.

3.3 Pour illustrer l'interpolation, dans le fichier *TypeScript* `etudiant.component.ts` définissez des nouvelles variables `nomEtu` et `prenomEtu` de type String qui représentera les noms et prénoms d'un étudiant. Utilisez ensuite ces variables pour les intégrer dans le fichier template `etudiant.component.html` Après ajustement de l'affichage il s'agira d'obtenir une page comparable à l'image suivante :



3.4 Le mécanisme d'interpolation accepte en fait toute expression de type *TypeScript* y compris l'appel de fonction. Dans le fichier *TypeScript* précédent, rajoutez deux fonctions `retNomEtu()` et `retPrenomEtu()` et replacez l'interpolation précédente dans le fichier template (`etudiant.component.html`) par l'appel de ces fonctions. Vérifiez que la page est toujours la même.

En dehors du mécanisme d'interpolation, le ***Property binding*** est un autre moyen d'intégrer des données de manière dynamique en modifiant directement les éléments du DOM. Pour illustrer ce mécanisme, nous allons changer l'état d'un bouton qui servira à changer le statut des étudiants.

3.5 Dans le fichier *TypeScript* `app.component.ts`, rajoutez une variable de type booléenne `estAuth` qui symbolisera le fait que l'utilisateur est authentifié ou non (l'initialiser à `false`). Dans le fichier template `app.component.html`, rajoutez un bouton *Tous présents*. Vous utiliserez un balise *button* avec la classe *Bootstrap* `"btn btn-success"` et vous rajouterez la propriété *disabled*.

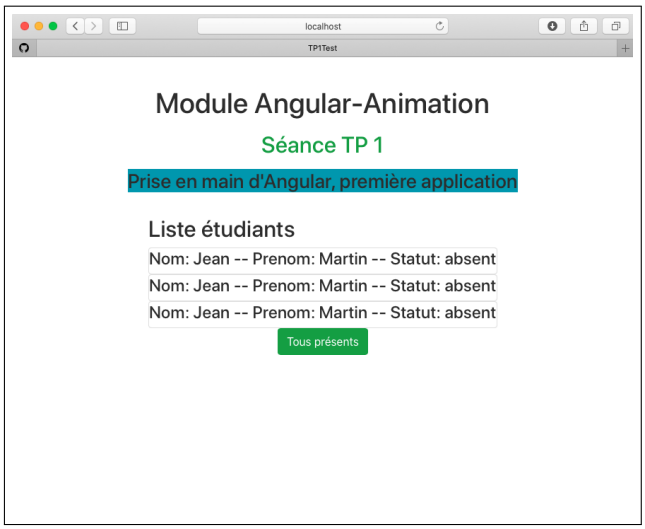
3.6 Pour effectuer la liaison entre une propriété et une variable, il faut placer la propriété entre crochets et la lier avec sa valeur booléenne donnée par une expression *TypeScript* mise dans une chaîne de caractères. Rendez le bouton actif uniquement si la variable `estAuth` est vraie. Testez le bon fonctionnement de la liaison en changeant manuellement la valeur de booléen de la variable `estAuth`.

3.7 Pour mettre en évidence que la liaison est bien dynamique, mettez la variable `isAuth` à *false* et rajoutez un constructeur (`constructor(){}`) dans la classe `AppComponent`. Dans ce constructeur changez la valeur de la variable `isAuth` en utilisant un délai de 3 secondes. Vous pourrez utiliser la fonction `setTimeout` avec la syntaxe suivante :

```
setTimeout( () => { }, delai_ms);
```

Un autre moyen d'ajouter des interactions dans l'interface est d'utiliser la liaison des événements. Dans la suite nous allons illustrer cette interaction avec l'évènement souris.

3.8 Dans la classe `etudiant.component`, rajoutez une nouvelle variable `statutEtu` de type `string` qui représentera le statut d'un étudiant et sera initialisé par défaut à "absent". Complétez l'affichage de l'étudiant de façon à faire apparaître son statut comme sur l'image suivante :



3.9 Pour faire une ***liaison par événement*** souris, il suffit d'utiliser des () avec à l'intérieur l'événement choisi (par exemple *click*) suivi du symbole = et d'une chaîne de caractères associée à une expression *TypeScript*. Dans le fichier `app.component.html`, rajoutez une liaison vers une nouvelle fonction `rendTousPres()` qui pour l'instant ne fera qu'afficher un message de test (avec `console.log()`). Vérifiez dans la console de votre navigateur que l'événement est bien produit.

Il peut aussi être pratique de rajouter des ***liaisons par des propriétés personnalisées***. Il est possible de rajouter une propriété personnalisée dans une classe simplement en rajoutant le mot clé @Input() avant sa déclaration. Ensuite lors de l'inclusion de l'objet dans le code HTML il faut préciser la valeur de la propriété dans une balise comme si c'était une propriété classique ayant pour nom la valeur de l'attribut de la classe (qui est précédé de @input())

3.10 Tester de rajouter une nouvelle propriété associées aux noms, prénoms et statut de l'étudiant. **Indication :** il est nécessaire de rajouter dans les imports votre fichier `etudiant.component.ts`, le module `Input`.

Partie IV: Pour ceux qui ont fini

Pour ceux qui ont fini, vous pourrez explorer les instructions utiles pour la structuration de documents (voir instructions en TP).